

EEE3096S

Embedded Systems II

Selection Process, Golden Measures, Intro to Benchmarking & and Code Reviews

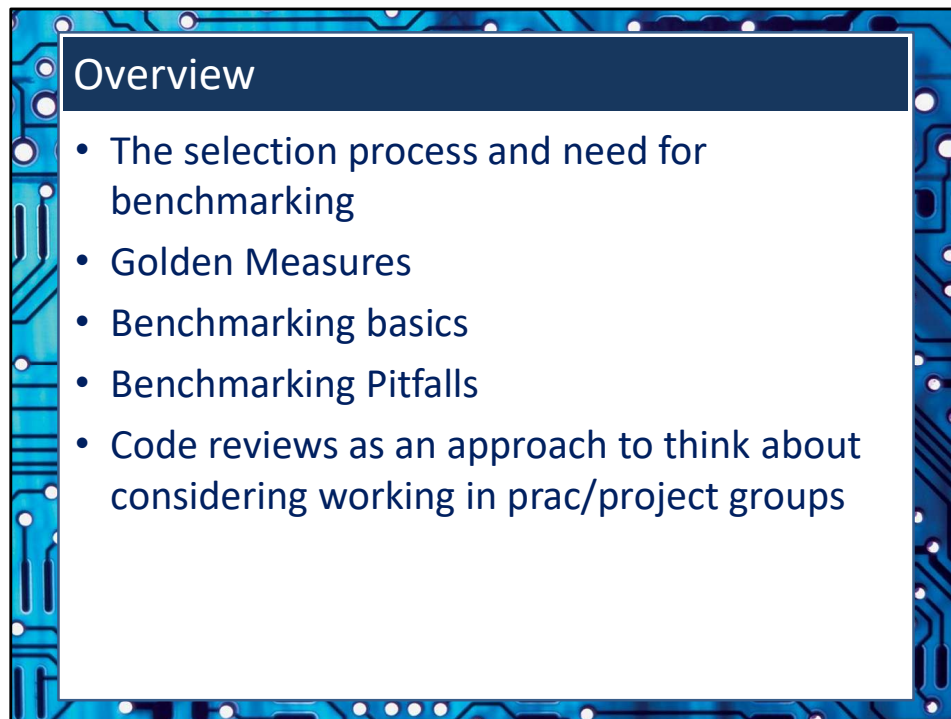
Lecture L04

Embedded Systems II

Dr Simon Winberg

Electrical Engineering
University of Cape Town

Welcome to the topic of Benchmarking. Coincidentally, these techniques are used through much of a professional industry project. Benchmarking to assess the performance of your system, possibly in comparison to other systems or to a defined baseline. Code reviews are things we will discuss later, and are linked to the topic of professional practice and benchmarking practice as a technique of having more than one individual scrutinize, quality inspect and critique each other's code or solutions. Techniques of benchmarking and code reviews are essential ingredients of a quality embedded systems development process. These tend to be done in some way irrespective of the development methodology, such as SCRUM or the Spiral Model, that may be in use.



This lecture starts off by discussing the important issue of the selection process and how it impacts the cost and sustainability of a product. Benchmarking is an important technique that is used in the selection process.

We then quickly move into the issues of benchmarking. And then on towards Code Reviews. After the selection process is made, and you have decided the hardware and tools to use, the technique of benchmarking typically carries on through the product. Code reviews and benchmarking are often done together or are interrelated. Hence the reasoning for delving directly into the concept and importance of code reviews after explaining benchmarking.

Selection Process

- **Selection Process =**
Selecting parts and other things from which you will build your embedded system
- This includes choosing processor(s), hardware components, software and code to incorporate and/or code or design aspects to reuse.



The Selection Process

- It's a significant part of embedded design process
- Involves selecting the best software and hardware **choices** for the required **functionality**
- Correct selection is **critical to profit !!**
- Can (greatly) lower production costs
 - No money wasted on unused features
- Lower non-recurring engineering costs
 - Development cheaper with good tools
- Better reliability and maintainability

The Selection Process.

Is A significant part of the embedded design process.

It Involves selecting the best software and hardware choices for the required functionality of a product.

It is about the Correct selection of parts being critical to the profit, as well as sustainability and quality of 1

The right selection of parts Can greatly lower production costs. And possibly production time as well.

It is partly about not wasting money on unused features or unnecessary pieces.

A large aspect is about Lower the non-recurring engineering costs. That is to say, time that the developmen

A lot of it is about how to get the Development cheaper. Which tends to mean using a good selection of to

Another important aspect of the selection process is choosing parts for Better reliability and maintainabilit

'Golden Measures'



Term: Golden Measure



- The **Golden Measure** concept:
 - A baseline benchmark solution (which is not about speed, but rather accuracy)
- The concept originates from theory of a 'gold standard measure'* which is:
 - A benchmark available under reasonable conditions. Not necessarily a perfect test, but the best available one that has a standard with known results. This is especially important when faced with the impossibility of direct measurements.

* This term originated more in medicine (although has a much older origin if looking further back) and is also widely applied to data science and computing nowadays.

Term: Golden Measure



- Avoid confusion ...
 - Do not confuse with notion of a perfect physical measures, e.g. 'perfect meter', which is about getting a perfect instance of the thing you want to compare against (if such a thing exists it makes the process or comparison and measure easier and very reliable).
(<https://www.atlasobscura.com/places/the-last-original-standard-metre-paris-france>)
- A golden measure in computing can be:
 - A solution that may run slowly, isn't optimized, but *you know* it gives (numerically speaking) excellent results

e.g., a solution written in MatLab, verify it is correct using graphs or exhaustive testing, or checking by hand with calculator, etc.

See example handout: [EEE3096S Lecture 04 Activity 1 -- Golden Measure.pdf](#)

Before we move on to benchmarking execution time, let's look at the A5 handout I passed around. On the front, there's a quick scenario for a geometric problem—calculating 2D distance for an embedded tracking application.

Take a second to look at the front side and ask yourself: what would your Golden Measure be here? Remember, the Golden Measure isn't about being fast or optimized; it's about absolute numerical correctness

Benchmarking

Benchmarking Basics



Let's get into the issue of benchmarking techniques.

Benchmarking

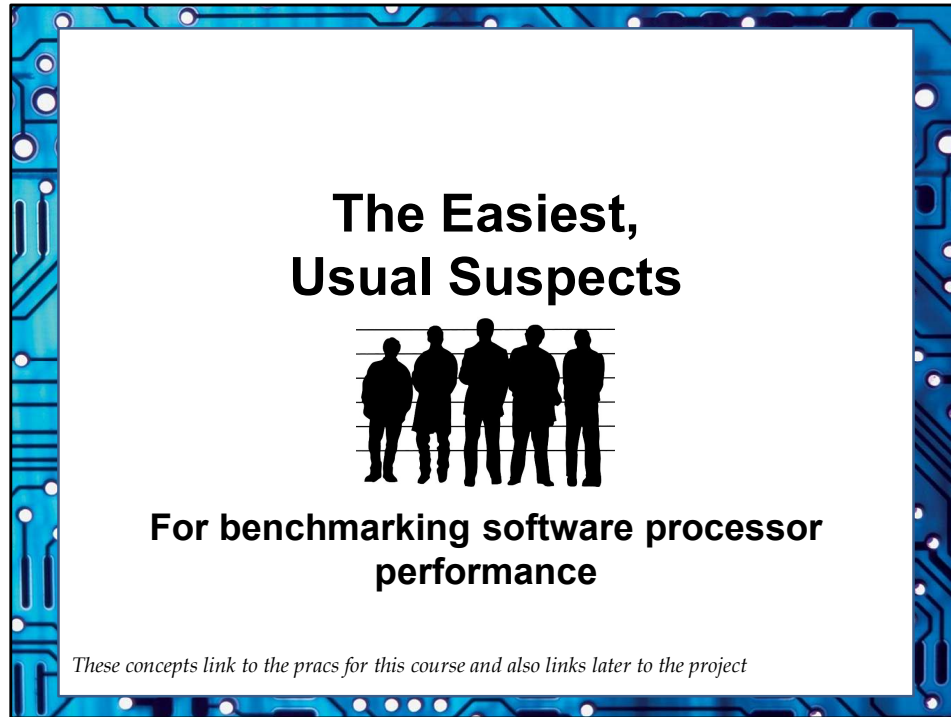


- **Benchmarking =**
Process of measuring performance of a system (an embedded computer system in our case)
- It's a program (or systematic approach) that **Quantitatively Evaluates** performance, cost and computer hardware and software resources (among other things) of a computing solution
- **Benchmark suite =**
A sets of benchmark programs designed to get a comprehensive view of the performance of a computer system for executing a variety of representative processing operations.

Benchmarking is the Process of measuring the performance of a system, an embedded computer system in our case.

It is a program, or systematic approach, that Quantitatively Evaluates the performance, cost and computer hardware and software resources of a computing solution.

A Benchmark suite is A sets of benchmark programs designed to get a comprehensive view of the performance of a computer system, for executing a variety of representative processing operations.



Now I will introduce the Easiest, or Usual Suspects, used for benchmarking software performance.

Wall Clock Time

- Generally the most accurate*: use a built-in timer, which is directly related to **real time** (e.g., if the timer measures 1s, then 1s elapsed in the real world)



Built-in timer with added features

- Technique:

```
unsigned long long start; // store start time
unsigned long long end;   // store end time
start = read_the_timer(); // e.g. time()
DO PROCESSING
end = read_the_timer();    // e.g. time()
.. Output the time measurement (end-start), or save
it to an array if printing will interfere with the
times. Note: to avoid overflow, used unsigned vars.
```

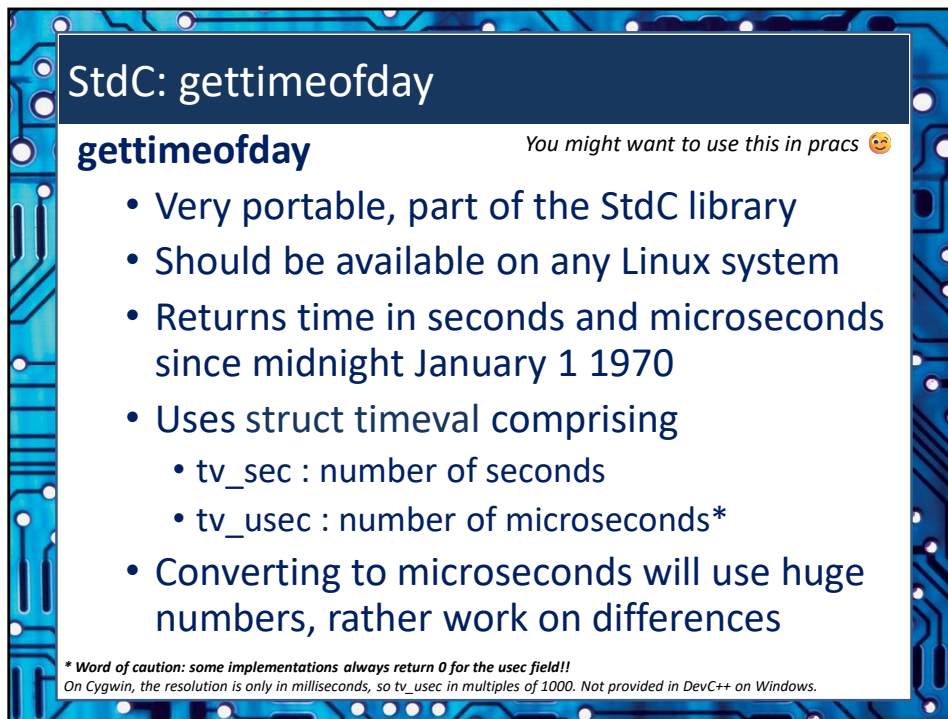
See file:
cycle.c
Cycle.h

* from measuring processor speed for an easy human perspective on the issue at least

Wall Clock Time.

Generally, an accurate measure for benchmarking is just Wall Clock Time. This is a term used in the embedded computing community. It just refers to time that is familiar to humans, in the real world. Like seconds ticking by on the office clock on the wall. Of course one is not likely to use just a physical watch or stopwatch. Rather it will be a time used that is internal to the processor or platform that you are using. Thus, this method is just using a built-in timer, which is directly related to real-time. If the timer measures 1 second, then 1 second elapsed in the real world.

The typical code for using this is to read the timer just before you start the process you want to time. And then read the timer just after the processing has completed. And that gives you the elapsed real time for doing the process of interest.



StdC: gettimeofday

gettimeofday *You might want to use this in pracs 😊*

- Very portable, part of the StdC library
- Should be available on any Linux system
- Returns time in seconds and microseconds since midnight January 1 1970
- Uses struct timeval comprising
 - tv_sec : number of seconds
 - tv_usec : number of microseconds*
- Converting to microseconds will use huge numbers, rather work on differences

* Word of caution: some implementations always return 0 for the usec field!!
On Cygwin, the resolution is only in milliseconds, so tv_usec in multiples of 1000. Not provided in DevC++ on Windows.

The standard C library's get time of day function provides a highly portable means to implement wall clock timing.

This function should be available on any Linux system.

The function is specified to return time in seconds and microseconds since midnight on January first, 1970. This moment in time is referred to as the Unix Epoch time. Unix development officially started in the year 1970, but not exactly on New Year's day that year. Anyway, this function returns a struct that provides accuracy and seconds and microseconds. But a word of caution is that some implementations do not return microsecond resolutions, only second resolution meaning you might not be able to do precise timing with this function.

gettimeofday example

```
#include <stdio.h>
#include <sys/time.h>
#include <time.h>
struct timeval start_time, end_time; // variables to hold start and end time

int main() {
    int tot_usecs; /* may want to use long int on CPU with smaller word length */
    int i,j,sum=0;
    gettimeofday(&start_time, (struct timezone*)0); // starting timestamp

    /* do some work */
    for (i=0; i<10000; i++)
        for (j=0; j<i; j++) sum += i*j;

    gettimeofday(&end_time, (struct timezone*)0); // ending timestamp

    tot_usecs = (end_time.tv_sec-start_time.tv_sec) * 1000000 +
                (end_time.tv_usec-start_time.tv_usec);
    printf("Total time: %d usec.\n", tot_usecs);
}
```



See [timing.c](#) code file

Here is some example code that is timed using the get time of day function. The process that we are timing is the for loop in the middle of the program. This example code is provided in the timing.c code file for this lecture.

Clock Wrapping

- While this issue isn't really a problem with `gettimeofday`, it is a problem when using a high-resolution time of limited length that might wrap (i.e. go from high value to low value) in the code section timed. Assuming that an unsigned integer is being used.
- The solution is doing the time duration calculation using AND or modula, e.g.:

`elapsed = (end - start) & MAXINT`

can use `UINT32_MAX = 0xFFFFFFFF` for 32-bit uint (or `uint32_t`)



Examples

Quick look at some code examples ...

Available on Amathuba .. EEE3096S Module 1:



A quick run through:
Timer.c Timer.h timer_example.c

EEE3096S-L04
Benchmarking

Timer (tic toc in
C)

solution_using
clock_ge_time

timer_wrapping

Benchmarking Pitfalls



Some common Benchmarking Pitfalls. In other words, things you want to look out for.

What is wrong about using only wall clock time?

It can provide a false impression of how effective your solution is – or rather more correctly it at least **doesn't give a 'full picture' of performance ...**



Recommended tips for using wall-clock time:

- Typically do tests after the system has 'warmed up' (cache loaded) by running the same data multiple times*
- May show the solution is quicker... but at what costs? e.g.:
 - Speed improved but accuracy sacrificed?
 - Development effort vs. execution speed improvement?
 - Resource costs for upgrading vs. costs saved by remaining with the old version?
 - Power usage? Does the new solution need more power (per execution, also on average including idle time)
 - Maintainability? (e.g. is the new version more complex?)
 - Environment impact? (Does the upgrade result in waste that could be environmentally detrimental)

* But note, this could also be a false impression depending how your system is likely to work, if operations are typically not going to be in cache when invoked, this approach could invalidate your benchmarking.

Is there anything wrong with using only wall clock time?

Well, yes. Proper benchmarking of an embedded system should do more than consider its runtime performance at one moment.

Looking just at the speed for a particular case can provide a false impression of how effective your overall solution is.

Or rather, more correctly, just using speed measurements does not give a 'full picture' of the system's performance.

A recommended tip for using wall-clock time is to do the timing tests after the system has 'warmed up'. As in after the CPU cache has loaded. This can be done by running the same data multiple times.

This method may show a solution that is quicker, than a cold start. But at what drawback? Here are some points to consider on this.

Warming up the case may show speed improvements, and allow platforms of a different type to be compared more fairly if you are only comparing the speed that instructions do their work. But this approach can lead to accuracy being sacrificed, because you are eliminating other aspects of the platform design from the analysis. Such as the time it takes for secondary memory to be accessed, and the cache to be loaded.

Other things to measure include:

Development effort, versus improvements in execution speed.

Resource costs for upgrading. Versus costs saved by remaining with one version.

Power usage? Does the enhanced solution need more power compared to a more basic version. And one should consider this in terms of per execution, and on average. Including idle time, when you would want the platform to use as little power as possible because it isn't doing anything needed.

And Maintainability? (e.g. is the fancier design more complex and thus more difficult to maintain?)

Don't forget the potential of Environment impact? For instance, does the upgrade result in waste that could be environmentally detrimental.

Benchmarking: what to test

- What can be benchmarked? For DSPs or embedded micros...
- Compiler
 - Converts High Level Language to Assembly language thus we benchmark compiler efficiency (such as how efficient is the generated assembly code?)
- The Processor
 - Code in hand-crafted/inspected assembly (to make comparisons fair)
- Operating System
 - Interrupt latencies, overhead of operating system calls, limits on devices, kernel size, availability of services and facilities such as support for virtual memory and paged memory.
- Platform
 - Scalability of memory. Peripheral limits. Interfaces supported. Power use. Power saving features. OS's supported.
- Applications

For more thorough Benchmarking, there are many things that could be tested.

Here is a list of some of the aspects that are commonly benchmarked in industry projects. These pertain to both DSP and embedded microprocessors.

Compiler options could be benchmarked. As in how efficient the code is that it generates. Not so much how efficiently the compiler runs, usually developers would far prefer a compiler that generates efficient time, albeit taking a while to do so. Than a compiler that runs quickly but generates inefficient code.

Processor options. Comparing processors to see which one works best for critical sections of code for the product being developed.

Benchmarking Operating System options. This involves testing operating systems, possibly just in terms of the operating systems functionality and its ease of use. Or analyzing how well it runs they run on the platform you want to use.

Benchmarking Platforms. Comparing things like how scalable their memory options are. How many peripherals they can connect to.

And Applications Benchmarking. There are a whole lot of things this may involve. One such is running different, often pre-coded, application types to see which provide the best solution for a particular type of need for your product. A product being developed could be a large system that incorporates various existing applications to do work needed for the product provided.

Benchmarking: what is usually measured

- For embedded and DSP systems we typically benchmark:
 - Cycle count
 - Data and Program memory usage
 - Execution time
 - Power consumption (has recently become a common thing to report on)

For embedded and DSP systems we typically benchmark these aspects.

Cycle count.

Data and Program memory usage.

Execution time.

Power consumption. (In recent years, the aspect of benchmarking power consumption has become a more common feature to report on, and may even be a necessary aspect for obtaining certifications in marketing or deploying the product).

Issue #1: Processor Performance

- This is usually talked about the most, and is often the most important design decision
- *Particularly:* is the processor/controller/DSP we're planning be fast enough?
- Performance measuring tools are available to analyze this, at least roughly, particularly:
 - MIPS (Million Instructions Per Second)
 - FLOPS (Floating Point MIPS)
 - Dhrystone Benchmark

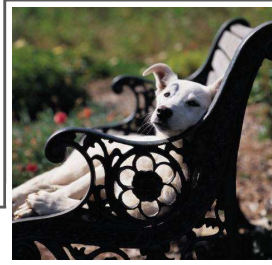
An important point to note: that processor manufacturer might optimize their toolchains to get better benchmark results using these common tools, so it's usually best to use some more representative tests for your particular application.



Examples of Standard Benchmarks

- The Dhrystone benchmark
 - Developed in 1984 by R. Wecker
 - Tests integer performance only
 - No I/O, no OS calls, no floating point
- Good for estimating integer performance
- Dhrystone score is the number of times a small program can run per second
- Like all benchmarks, applicability of dhrystone varies (may be totally irrelevant to your specific application)

... so don't get too comfortable using a common very 'general purpose' benchmarking tool like Dhrystone!



Meaningful Benchmarking

- How will the processor perform with our application?
- Including complex factors
 - Interrupt handling
 - Task switching
 - Memory performance
- Even more important for Real Time applications
 - Context switching time is critical
 - Save registers, change mode, etc.

EEMBC

Not Examined

- EDN* Embedded Micro Benchmark Consortium
- 46 test split into 5 suites
 - Automotive, consumer, networking, office automation, telecommunications
- Advantages
 - Speed and size statistics
 - Tailored to real application domains
 - Well designed, tested and documented
- Disadvantages
 - Expensive

* EDN: *Electronics Design News* publication and information company
See: www.edn.com

Running Custom Benchmarks

- Typically use evaluation boards
 - Save costs of designing support hardware
- Evaluation boards
 - Single board computers
 - Include a selection of common peripherals
- Some manufacturers sell **Hot Boards**
 - Specifically designed to boost benchmarks
 - Watch out for these boards



Bad News About Benchmarks

- Benchmarks do not consider all design factors which have an effect on performance, such as:
 - Power consumption
 - Physical size
 - Board layout
- Do not rely entirely on benchmark results for processor selection

Your benchmark results may look high and mighty – and sit above the rest of flock – but in reality you might just have seen part of the story; it may spend most time lower down ☺



Thinking Activity / Test Prep.

Consider that you are wanting to use an embedded operating system for a new product.



New product

Q: What are some of the factors that would influence your choice of operating system?

Consider discussing this in small groups or with a study buddy.

Strategy Tips: Need to think of the functionality and components involved, and from there decide OS needs, performance needs, etc.

I'll share some insight on this next week when you've hopefully thought over it on your own.



An ingredient of a professional engineer

EEE3095S (CS) Students
Take Note re Project!

The Code Review



*These are things you should be aware of, and helps with thinking
and being prepared for pracs or projects
(there is no EEE3096S project, but is a **project for EEE3095S!**)*

What is a Code Review?

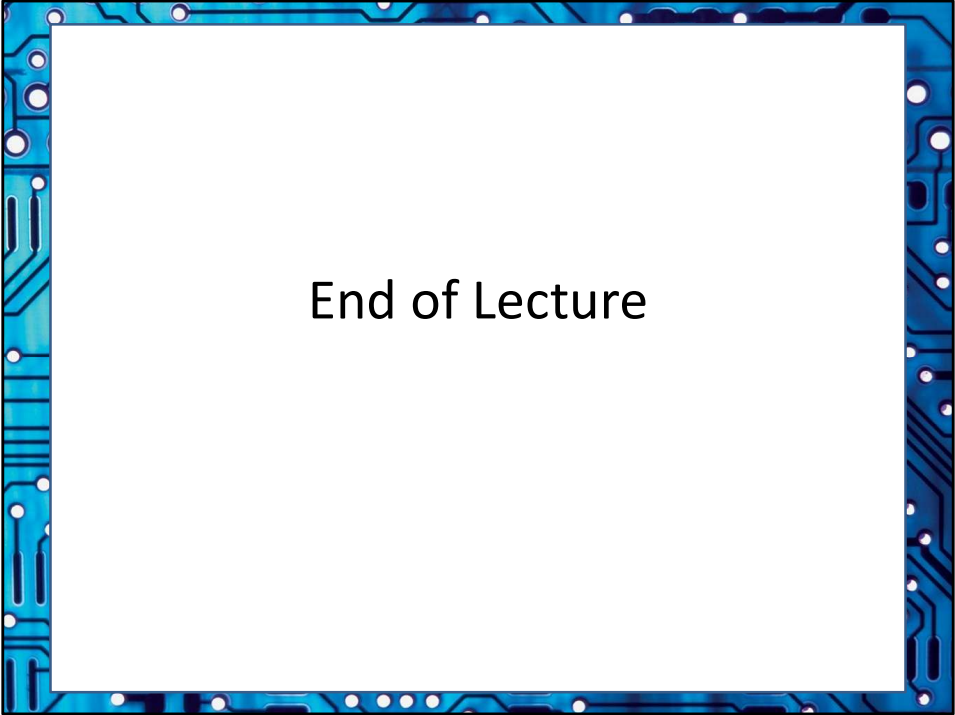
- It is a practice (e.g. meeting or regular team-link up) of reviewing code developed for a project
- It follows (generally) a systematic practice
(formulate your own initiative method for now!)
- It's about managing quality of code and ways to assess this, e.g. consider:
 - How to spot and log any defects
 - How many or common are the defects
 - Methods to use, such as (not necessarily all of these, not necessarily used in every review) are:
 - Unit testing
 - Function testing
 - Integration testing
 - How can this be improved?

Why Mention Code Reviews?



A best practice, commonly used in industry

Might also improve the mark you get for assignments



End of Lecture